HW 03 — Numerical Integration and Root-Finding: Quantum Levels of O_2

Overview

This assignment builds two general-purpose numerical tools — an adaptive integrator and a bracketed root-finder — and applies them to compute the quantized vibrational energy levels of the O_2 molecule using the **Bohr-Sommerfeld-Wilson (BSW)** quantization rule in both the harmonic approximation and the full Lennard-Jones potential.

Task 1: Numerical Integration — `f_integral`

Problem Statement

Implement `f_integral(f, a, b, eps, global_type, integral_type)` that computes $\int_a^b f(x)\,dx$ using two quadrature rules (`simpson`, `gauss`) and two global strategies (`fix_segments`, `recursive`). Validate on $\int_0^\pi \sin x\,dx = 2$.

The Two Quadrature Rules

Both rules approximate $\int_a^b f(x)\,dx$ on a single small interval.

Simpson's rule: Approximates f by a parabola through the left endpoint, midpoint, and right endpoint:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}\left[\frac{f(a)}{3} + \frac{4f(m)}{3} + \frac{f(b)}{3}\right], \quad m = \frac{a+b}{2}$$

Uses 3 function evaluations. Truncation error $O(h^5)$ per interval, $O(h^4)$ globally.

3-point Gauss-Legendre rule: Places evaluation points at $\pm\sqrt{3/5}$ and 0 on $[-1, 1]$, then maps to $[a, b]$:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}\left[\frac{5}{9}f(x_1) + \frac{8}{9}f(x_2) + \frac{5}{9}f(x_3)\right]$$

where $x_1 = m - \sqrt{0.6}\cdot h$, $x_2 = m$, $x_3 = m + \sqrt{0.6}\cdot h$. Also 3 evaluations, but the node placement is optimal: Gauss-Legendre with n points integrates polynomials of degree $\leq 2n-1$ exactly. For $n = 3$ this is degree 5, same order as Simpson — but with a smaller constant.

Implementation of a single segment:

```
def _integrate_segment(f, a, b, integral_type):
    mid = 0.5 * (a + b);  half = 0.5 * (b - a)
    if integral_type == 'simpson':
        return half * (f(a)/3 + 4*f(mid)/3 + f(b)/3)
    elif integral_type == 'gauss':
        sq = np.sqrt(0.6)
        return half * (5/9*f(mid - sq*half) + 8/9*f(mid) + 5/9*f(mid + sq*half))
```

The Two Global Strategies

`fix_segments`: Divides $[a, b]$ into $n = \lfloor 1/\varepsilon \rfloor$ equal subintervals and applies the quadrature rule to each. Work is fixed regardless of where the integrand varies.

```
n = max(1, int(1.0 / eps))
xs = np.linspace(a, b, n + 1)
total = sum(_integrate_segment(f, xs[i], xs[i+1], integral_type) for i in range(n))
```

`recursive` (adaptive): Starts with the whole interval. On each sub-interval, compares the one-piece estimate I_{full} against the two-piece estimate I_{half}. If $|I_{\text{full}} - I_{\text{half}}|$ exceeds the local tolerance (proportional to the sub-interval length), it subdivides and recurses. This concentrates work where the integrand is varying rapidly.

```
def _recurse(lo, hi, depth):
    I_full = _integrate_segment(f, lo, hi, integral_type)
    mid = 0.5*(lo + hi)
    I_half = _integrate_segment(f, lo, mid, integral_type) + _integrate_segment(f, mid, hi, integral_ty
    tol = eps * (hi - lo) / full_length
    if depth >= 50 or (depth >= 2 and abs(I_full - I_half) <= tol):
        return I_half
    return _recurse(lo, mid, depth+1) + _recurse(mid, hi, depth+1)
```

Results: `sin(x)` from 0 to π

Strategy	Rule	Error	Function calls
fix_segments	simpson	$\sim 10^{-16}$	30 000
fix_segments	gauss	$\sim 10^{-16}$	30 000
recursive	simpson	$\sim 10^{-6}$	135
recursive	gauss	$\sim 10^{-9}$	63

`recursive+gauss` achieves the best accuracy with the fewest calls because Gauss nodes are optimally placed and the adaptive strategy avoids over-sampling the smooth $\sin x$ integrand.

Task 2: Root-Finding — `x_root`

Problem Statement

Implement `x_root(f, a, b, epsx, epsf, type)` that finds a root of f in $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs. Implement `bisection` and `secant` methods, stopping when the bracket narrows to `epsx` or $|f(x)| < \varepsilon_f$.

Bisection Method

Algorithm: Repeatedly halve the bracket. If $f(\text{mid})$ and $f(a)$ have the same sign, the root is in $[\text{mid}, b]$; otherwise it's in $[a, \text{mid}]$.

```
while True:
    mid = 0.5 * (lo + hi)
    fmid = f(mid)
    if abs(fmid) < epsf or (hi - lo) < epsx:
        return mid
```

```
    if flo * fmid < 0:
        hi = mid
    else:
        lo, flo = mid, fmid
```

Convergence: Linear — each step halves the interval width. Guarantees convergence but is slow: needs $\sim \log_2((b-a)/\varepsilon_x)$ steps.

Secant Method (with bisection fallback)

Algorithm: Fits a straight line through the last two points (x_0, f_0) and (x_1, f_1) and extrapolates to find the next estimate:

$$x_2 = x_1 - f_1 \cdot \frac{x_1 - x_0}{f_1 - f_0}$$

If the result lies outside the current bracket $[lo, hi]$, it falls back to bisection. The bracket is updated so it always contains the root.

```
x2 = x1 - f1*(x1 - x0)/(f1 - f0)
if x2 < lo or x2 > hi:           # unsafe: use bisection
    x2 = 0.5*(lo + hi)
f2 = f(x2)
if flo * f2 < 0: hi, fhi = x2, f2
else:            lo, flo = x2, f2
x0, f0 = x1, f1;  x1, f1 = x2, f2
```

Convergence: Superlinear (order ≈ 1.618, the golden ratio) when well-behaved. The bisection fallback prevents the secant from escaping the bracket.

Result on $\cos(x) = 0$ **in** $[0.1, \pi/2 + 0.1]$**:** Bisection needs 33 calls; secant needs only 7.

Task 3: Harmonic Oscillator Verification

Problem Statement

For $v(x) = x^2$ and $\gamma = 1$, the BSW quantization condition is:

$$\gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v(x)}\, dx = \left(n + \frac{1}{2}\right) \pi$$

where $x_{1,2} = \mp\sqrt{\varepsilon}$ are the classical turning points. This must be solved for ε_n for $n = 0, 1, 2, 3, 4$.

For the harmonic oscillator, the exact answer is $\varepsilon_n = 2n + 1$.

How It Works

Define $s(\varepsilon) = \gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v(x)}\, dx$ and root-find $f(\varepsilon) = s(\varepsilon) - (n + 1/2)\pi = 0$.

The integrand $\sqrt{\varepsilon - x^2}$ has **integrable square-root singularities** at the turning points (where it equals zero). To avoid roundoff at exactly the turning point, the integration limits are shifted inward by $\delta = 10^{-9}(x_2 - x_1)$:

```
ff = lambda x: np.sqrt(max(0., energy - v_sq(x)))
return gamma * f_integral(ff, x_1 + delta, x_2 - delta, 1e-6, 'recursive', 'gauss')
```

Result: Relative errors $< 10^{-9}$, confirming the code reproduces the known harmonic-oscillator energies $\varepsilon_n = 1, 3, 5, 7, 9$.

Task 4: Harmonic Approximation to the Lennard-Jones Potential

Near the minimum $x_{\min} = 2^{1/6}$, the LJ potential is approximated by a parabola:

$$v_{\text{hr}}(x) = -1 + \frac{1}{2}k(x - x_{\min})^2, \quad k = v''(x_{\min}) = 4\left(\frac{156}{x_{\min}^{14}} - \frac{42}{x_{\min}^{8}}\right)$$

The turning points are $x_{1,2} = x_{\min} \mp \sqrt{2(\varepsilon + 1)/k}$. With $\gamma = 150$, this yields 14 bound states with nearly **constant spacing** ≈ 0.0713 (as expected for a harmonic well).

Task 5: Full Lennard-Jones Potential

$$v(x) = 4\left(\frac{1}{x^{12}} - \frac{1}{x^{6}}\right)$$

The exact turning points for energy ε are solved analytically from $v(x) = \varepsilon$:

$$x_{1,2} = \left(\frac{2}{1 \pm \sqrt{1 + \varepsilon}}\right)^{1/6}$$

With $\gamma = 150$, the code finds 39 bound states. Key observations: - Level spacing **decreases** at high n (real well becomes softer near $\varepsilon = 0$). - The harmonic approximation matches well for low n (small-amplitude oscillations near the minimum) but diverges at high n where the asymmetry of the true potential matters. - More function calls are needed for high-n states because the outer turning point moves far out into the flat part of the potential.

Task 7: Effect of Accuracy Parameter on ε_{18}

Repeats the LJ calculation for $n = 18$ with `ep` ranging from 10^{-3} to 10^{-12}. The integration tolerance is set to $10^4 \cdot$ ep and the root tolerance to `ep`.

Result: The computed ε_{18} converges to ≈ -0.1665468 as `ep` decreases. Below `ep` $\approx 10^{-9}$, the answer is stable but each tighter tolerance costs significantly more function calls. This illustrates the trade-off: accuracy is not free, and there is a point of diminishing returns where further refinement is not worth the cost.

Task 8: Comparison with SciPy

Repeats Task 5 using `scipy.integrate.quad` and `scipy.optimize.brentq` in place of the custom functions.

`quad` uses adaptive Gaussian quadrature with built-in error control. `brentq` is a hybrid of bisection, secant, and inverse quadratic interpolation. The differences between the custom and SciPy results are $\lesssim 10^{-7}$, confirming the custom implementation is correct. SciPy typically uses fewer function calls because its error estimators are more refined.

Key Takeaways

- **Gauss-Legendre beats Newton-Cotes per function call.** With n points, Gauss integrates polynomials up to degree $2n-1$ exactly (degree 5 for 3 points), versus degree 3 for 3-point Simpson — same cost, smaller error.
- **Adaptive (recursive) quadrature** concentrates effort where the integrand varies, comparing a one-piece against a two-piece estimate and subdividing only when they disagree. It is far cheaper than fixed sampling for smooth or locally-varying functions.
- **Bisection is slow but bulletproof** (guaranteed linear convergence); the **secant method** is superlinear (order ≈ 1.618) but can escape the bracket — so a robust solver uses secant with a **bisection fallback**.
- **A bracketed root requires a sign change** $f(a)f(b) < 0$; the bracket must be maintained so it always contains the root.
- **Integrable singularities** (here $\sqrt{\varepsilon - v}$ at the turning points) are handled by nudging the integration limits inward by a tiny δ and letting the adaptive integrator resolve the steep endpoint behavior.
- **The BSW quantization rule** $\gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v}\, dx = (n + \frac{1}{2})\pi$ turns "find the energy levels" into a root-find-inside-an-integral — a recurring pattern in computational physics.
- **Tolerance has diminishing returns:** past a point, tightening it only multiplies the cost without changing the answer. Always validate a custom routine against a trusted library (`quad`, `brentq`).